

Module 17: Standard Library Tour

Four corners of the library you will use constantly, chosen because each has a trap that has cost people real time.

Every example here is **reproducible**: fixed clocks, fixed dates, seeded random generators. That is partly so the lesson is stable, and partly because making time and randomness reproducible is itself the technique you need for testing.

What You'll Learn

- The String methods worth memorising, and `formatted`
- `java.time`: which type to pick, and why its arithmetic clamps
- Daylight saving, where date handling actually breaks
- Five `Math` behaviours that have produced real bugs
- Reproducible random numbers, and when never to use them

Coming From Another Language

	Python	Java
Format	f-strings	<code>"%s".formatted(x)</code>
Date	<code>datetime</code>	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code>
Timezone-aware	opt-in via <code>tzinfo</code>	opt-in via the type you choose
Duration	<code>timedelta</code>	<code>Duration</code> and <code>Period</code> , and they differ
Random	<code>random.seed(42)</code>	<code>RandomGeneratorFactory.of(...).create(42)</code>

The row that matters: Java makes you pick a type that either carries a zone or does not.

`LocalDateTime` is **not** a moment in time, and treating it as one is the most common `java.time` bug.

The Lesson

Strings

From `Strings.java`:

```

" x ".strip()           // Unicode-aware; trim() is not
"a,b,c".split(",")      // takes a REGEX, not a literal
String.join("-", "2026", "09")
"ab".repeat(3)
"%-10s|%8.2f|%,d".formatted("name", 3.14159, 1_234_567)

```

Two traps:

- **split** takes a regex. Splitting on `.` or `|` without escaping gives the wrong answer, and it is a classic one-hour bug.
- **compareTo** returns a negative number, zero, or a positive one. Not `-1`, `0`, `1`. Code that assumes otherwise is wrong.

Format conversions worth memorising: `%s` anything, `%d` integer, `%.2f` two decimal places, `%n` platform newline, `%,d` thousands separator, `%%` a literal percent. `%10s` right-aligns, `%-10s` left-aligns.

java.time: choosing the type

Type	Holds	Use for
<code>LocalDate</code>	date only	a birthday
<code>LocalTime</code>	time only	opening hours
<code>LocalDateTime</code>	both, no zone	a wall clock reading
<code>ZonedDateTime</code>	a real moment somewhere	a meeting
<code>Instant</code>	a point on the UTC timeline	a timestamp
<code>Duration</code>	hours, seconds	elapsed time
<code>Period</code>	years, months, days	calendar amounts

`LocalDateTime` has no zone. `2026-09-23T10:15` is a moment in some unstated place. Storing one and assuming UTC is the bug this module most wants you to avoid.

Every `now()` takes an optional `Clock`. Code calling `now()` with no argument cannot be tested for "what happens on 29 February". Take a `Clock` as a dependency and the problem disappears.

Arithmetic that clamps

From `DateAndTime.java`:

```

31 Jan + 1 month: 2026-02-28
29 Feb + 1 year:  2025-02-28
and back again:   2026-01-28
so +1 month then -1 month is NOT the identity.

```

Both clamp to the last valid day rather than rolling forward. That is deliberate, and it is **not reversible**. If your billing code adds a month and subtracts one, it does not land where it started.

`Duration` is time-based and exact. `Period` is calendar-aware, so one month is 28, 29, 30 or 31 days depending on where you apply it. That is why both exist.

Always pass a `Locale` to `ofPattern`. Without one it uses the machine default, so the same code formats differently on a server abroad. Same class of bug as the default charset before Java 18.

Daylight saving

From `TimeZonesAndDst.java`:

```
--- the spring forward gap ---
01:30 EST:      2026-03-08T01:30-05:00[America/New_York]
plus one hour:  2026-03-08T03:30-04:00[America/New_York]
02:30 does not exist; atZone gives: 2026-03-08T03:30-04:00[America/New_York]

--- the autumn fall-back overlap ---
first  (EDT): 2026-11-01T01:30-04:00[America/New_York]
second (EST): 2026-11-01T01:30-05:00[America/New_York]
```

Three things happen there:

1. Adding one hour to 01:30 gives **03:30**, because 02:00 never existed. The arithmetic is right; one real hour passed.
2. A local time inside the gap does not exist. Java does not throw, it moves forward.
3. In autumn, 01:30 happens **twice**. `atZone` picks the earlier offset. If you need the later one you must say so, and silently assuming either is how bookings get double-counted.

And the distinction that matters most:

`plusDays(1)` keeps the wall-clock time. `plusHours(24)` adds 24 real hours. On two days a year they give different answers. "Same time tomorrow" and "twenty-four hours from now" are different operations.

The rules that keep you safe: store an `Instant` or UTC and convert only for display, use a region `ZoneId` like `Europe/London` rather than a fixed offset, and never store a `LocalDateTime` for a real moment.

Math

From `MathSurprises.java`:

```

Math.abs(MIN_VALUE):    -2147483648
is it negative?         true
  abs(MIN_VALUE) % 7    = -2    <- a negative array index
  floorMod(MIN_VALUE, 7) = 5    <- always in range

-7 / 2                  = -3
Math.floorDiv           = -4

Math.round(2.5)         = 3
Math.round(-2.5)        = -2

MAX_VALUE + 1           = -2147483648
Math.addExact throws: integer overflow

```

abs can return a negative number. The `int` range is asymmetric, so negating the minimum wraps back to itself. This matters because `Math.abs(hash) % buckets` is a common idiom that produces a **negative index**, as the output above shows. Use `Math.floorMod(hash, buckets)` instead, which is always in range, or `Math.absExact` which throws.

Note the example uses 7 buckets, not 8. `MIN_VALUE` is divisible by 8, so with 8 both forms return `0` and the bug stays hidden. Choosing an example where the failure actually appears is part of the work.

Integer division truncates toward zero; floorDiv rounds toward negative infinity. They differ for negatives.

round rounds half toward positive infinity, so `-2.5` becomes `-2`, not `-3`.

Overflow is silent. `addExact`, `subtractExact` and `multiplyExact` refuse to wrap. Use them wherever a wrapped value is worse than a crash, which for anything counting money or memory is always.

Random numbers

From `RandomNumbers.java`:

```
RandomGenerator rng = RandomGeneratorFactory.of("L64X128MixRandom").create(42);
```

Note the shape: the **factory** creates the generator. `RandomGenerator.of(name)` returns a generator directly and has no `create`.

Same seed, same sequence. A test that shuffles or samples needs a fixed seed, or it fails once a month for reasons nobody can reproduce. Take the generator as a parameter, exactly as you would take a `Clock`.

`nextInt(bound)` excludes the bound; `nextInt(low, high)` includes low and excludes high.

Never use these for anything security-relevant. They are pseudo-random and predictable given enough output. Use `java.security.SecureRandom` for tokens, session ids and keys.

Do not share one generator across threads. `java.util.Random` is synchronised and becomes a bottleneck; `ThreadLocalRandom.current()` gives each thread its own.

Run It

```
java modules/17-standard-library-tour/examples/Strings.java
java modules/17-standard-library-tour/examples/DateAndTime.java
java modules/17-standard-library-tour/examples/TimeZonesAndDst.java
java modules/17-standard-library-tour/examples/MathSurprises.java
java modules/17-standard-library-tour/examples/RandomNumbers.java

./scripts/verify-examples.sh modules/17-standard-library-tour
```

Run any of them twice and the output is identical, because none depends on the current time or an unseeded generator.

Common Mistakes

Storing a `LocalDateTime` for a real moment. It has no zone. Store an `Instant`.

Using a fixed offset instead of a region id. `+01:00` is wrong for half the year in London.

Calling `now()` with no `Clock`. The code becomes untestable for any date-dependent branch.

`ofPattern` without a `Locale`. Different output on a server abroad.

`Math.abs(hash) % n` for a bucket index. Negative once in four billion. Use `floorMod`.

Comparing doubles with `==`. Compare within a tolerance, or avoid floating point for money.

An unseeded generator in a test. A flake once a month that nobody can reproduce.

`split(".")`. That is a regex matching any character.

Key Takeaways

- **Pick the `java.time` type by whether it needs a zone.** `LocalDateTime` does not carry one.
- **Take a `Clock` and a `RandomGenerator` as dependencies,** so time and randomness become testable.
- **Date arithmetic clamps and is not reversible.**
- **`plusDays(1)` and `plusHours(24)` differ** on daylight saving days.
- **`abs` can be negative, division truncates, `round` is asymmetric, overflow is silent.** The `Exact` methods throw instead.
- **Seed generators for tests; use `SecureRandom` for anything that matters.**

Homework

[homework/README.md](#)

Build a scheduling tool that survives daylight saving, then reproduce four `Math` traps. Reference solution in [solutions/17-standard-library-tour/](#).